

Homework 3

MTH312: Data Science Lab 3

Group 5

2026-03-10

true

QUESTION1

Introduction

This report investigates whether the `Boston` dataset originates from a multivariate normal (MVN) distribution using data depth methodologies. Data depth provides a center-outward ordering of multidimensional data, allowing us to generalize concepts like quantiles to higher dimensions.

To conduct a rigorous analysis, this study employs a two-pronged approach:

1. **Visual Inspection:** Utilizing Depth-Depth plots (DD-plots) across five distinct depth measures (Mahalanobis, Spatial, Halfspace, Simplicial, and Simplicial Volume) to check for global deviations from normality.
2. **Formal Hypothesis Testing:** Implementing a custom bootstrap-based Goodness-of-Fit test utilizing the Kolmogorov-Smirnov (KS) and Cramer-von Mises (CvM) statistics on depth values.

Part 1: Visual Inspection via DD-Plots

A DD-plot graphs the depth of the sample data points relative to the sample itself against their depth relative to a theoretical reference distribution (in this case, a simulated MVN distribution with the same mean and covariance as the sample). If the sample data follows the theoretical distribution, the points should align closely along the $y = x$ diagonal.

```
# Load Required Libraries
library(ddalpha)
library(ggplot2)
library(gridExtra)
library(MASS)

# 1. Clean Data & Calculate parameters
data(Boston)
n <- dim(Boston)[1]
ind <- sample(1:n,100)
boston_clean <- as.matrix(Boston[ind, c("crim","rm","medv")])
mu <- colMeans(boston_clean)
sigma <- cov(boston_clean)

# 2. Simulate multivariate normal data with matching parameters
set.seed(123)
sim_normal <- mvrnorm(n = nrow(boston_clean), mu = mu, Sigma = sigma)

# Combine datasets for depth evaluation
Z <- rbind(boston_clean, sim_normal)

# Helper function for plotting
create_dd_plot <- function(depth_data, depth_normal, title) {
  df <- data.frame(Data_Depth = depth_data, Normal_Depth = depth_normal)
  ggplot(df, aes(x = Data_Depth, y = Normal_Depth)) +
    geom_point(alpha = 0.6, color = "darkblue") +
    geom_abline(slope = 1, intercept = 0, color = "red", linetype = "dashed", linewidth = 1)
  +
    ggtitle(title) +
    theme_minimal() +
    labs(x = "Depth w.r.t Iris Data", y = "Depth w.r.t Normal Data")
}

plots <- list()

# Calculate Depths
# Mahalanobis
d_data_mah <- depth.Mahalanobis(Z, boston_clean)
d_norm_mah <- depth.Mahalanobis(Z, sim_normal)
plots[[1]] <- create_dd_plot(d_data_mah, d_norm_mah, "Mahalanobis Depth")

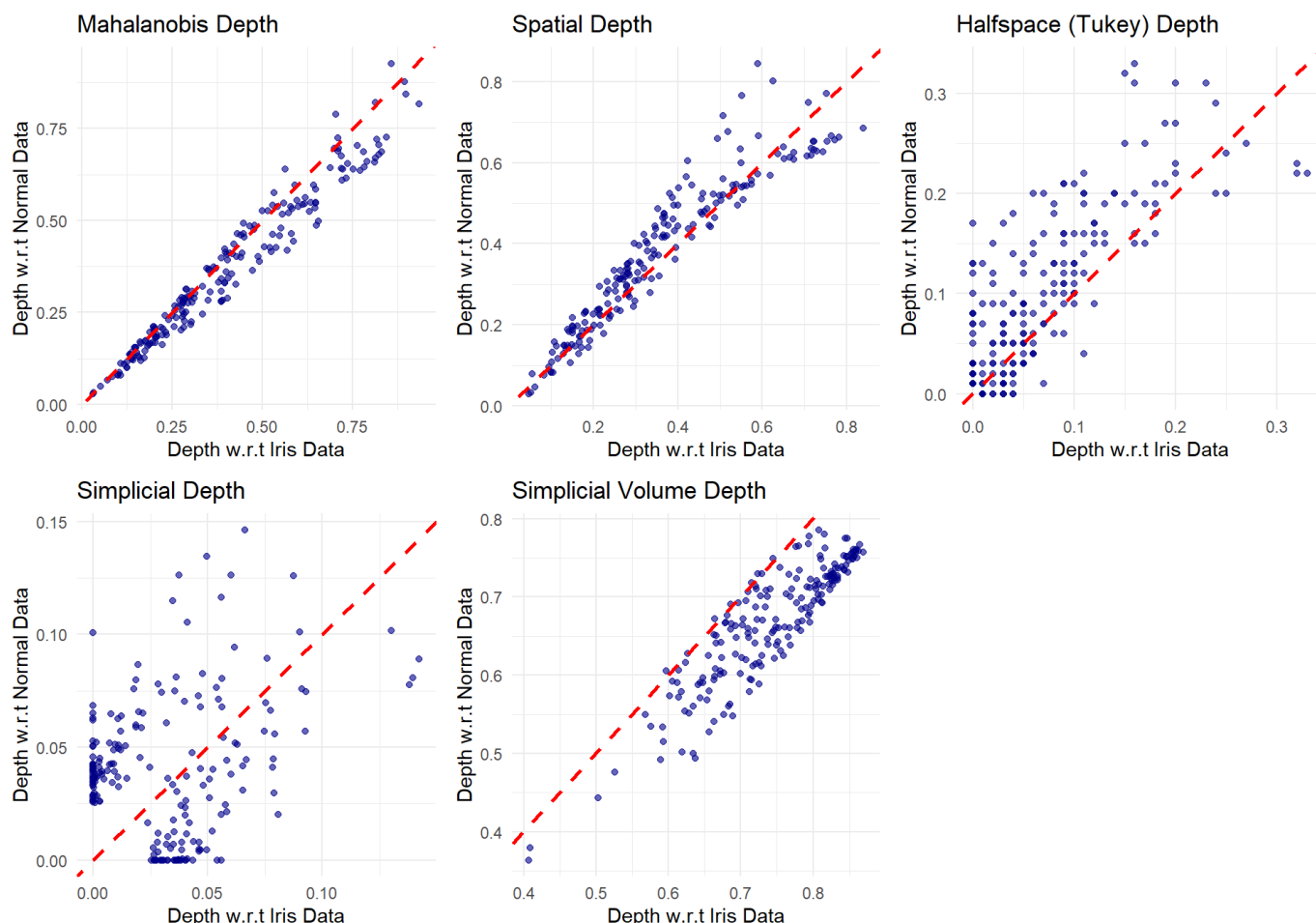
# Spatial
d_data_spa <- depth.spatial(Z, boston_clean)
d_norm_spa <- depth.spatial(Z, sim_normal)
plots[[2]] <- create_dd_plot(d_data_spa, d_norm_spa, "Spatial Depth")

# Halfspace (Tukey)
d_data_hs <- depth.halfspace(Z, boston_clean, exact = FALSE)
d_norm_hs <- depth.halfspace(Z, sim_normal, exact = FALSE)
plots[[3]] <- create_dd_plot(d_data_hs, d_norm_hs, "Halfspace (Tukey) Depth")

# Simplicial
d_data_sim <- depth.simplicial(Z, boston_clean, exact = FALSE)
d_norm_sim <- depth.simplicial(Z, sim_normal, exact = FALSE)
plots[[4]] <- create_dd_plot(d_data_sim, d_norm_sim, "Simplicial Depth")
```

```
# Simplicial Volume
d_data_sv <- depth.simplicialVolume(Z, boston_clean, exact = FALSE)
d_norm_sv <- depth.simplicialVolume(Z, sim_normal, exact = FALSE)
plots[[5]] <- create_dd_plot(d_data_sv, d_norm_sv, "Simplicial Volume Depth")

# Display all plots
grid.arrange(grobs = plots, ncol = 3)
```



Visual Conclusion

The DD-plots above are calculated using the columns 'crim', 'rm' and 'medv' of the Boston dataset. The points in the DD-plots exhibit severe bowing and deviate significantly from the red dashed $y = x$ reference line all depth methods apart from mahalanobis. Visually, the dataset does **not** follow a multivariate normal distribution.

Part 2: Formal Goodness-of-Fit Testing

Visual tests are subjective. To rigorously test for normality, we establish the following hypotheses:

- * **Null Hypothesis (H_0):** The data follows a multivariate normal distribution.
- * **Alternative Hypothesis (H_1):** The data does not follow a multivariate normal distribution.

Since we are estimating the mean and covariance from the sample data itself, standard critical values for Kolmogorov-Smirnov (KS) and Cramer-von Mises (CvM) tests are overly conservative. To correct this, we use a parametric bootstrap approach to simulate the true null distribution of the test statistics.

We will run this formal test on the dataset to see if the underlying population is normal.

1. KS Statistic Function

```
get_ks_stat <- function(depths_sample, depths_ref) {
  F_ref <- ecdf(depths_ref)
  F_samp <- ecdf(depths_sample)
  grid <- sort(unique(c(depths_sample, depths_ref)))
  max(abs(F_samp(grid) - F_ref(grid)))
}
```

2. CvM Statistic Function

```
get_cvm_stat <- function(depths_sample, depths_ref) {
  n <- length(depths_sample)
  F_ref <- ecdf(depths_ref)
  sorted_samp <- sort(depths_sample)
  U <- F_ref(sorted_samp)
  sum((U - (2 * (1:n) - 1) / (2 * n))^2) + 1 / (12 * n)
}
```

3. Main Bootstrap Test Function

```
depth_mvn_test <- function(data, method, B, alpha = 0.05) {
  data <- as.matrix(data)
  n <- nrow(data)

  mu_hat <- colMeans(data)
  Sigma_hat <- cov(data)

  set.seed(123)
  n_ref <- max(2000, 10 * n)
  Z_ref <- mvrnorm(n_ref, mu_hat, Sigma_hat)

  d_sample <- depth.(data, Z_ref, notion = method, exact=FALSE)
  d_ref <- depth.(Z_ref, Z_ref, notion = method, exact=FALSE)

  ks_obs <- get_ks_stat(d_sample, d_ref)
  cvm_obs <- get_cvm_stat(d_sample, d_ref)

  ks_boots <- numeric(B)
  cvm_boots <- numeric(B)

  for (b in 1:B) {
    X_boot <- mvrnorm(n, mu_hat, Sigma_hat)
    mu_boot <- colMeans(X_boot)
    Sigma_boot <- cov(X_boot)

    Z_ref_boot <- mvrnorm(n_ref, mu_boot, Sigma_boot)

    d_samp_boot <- depth.(X_boot, Z_ref_boot, notion = method, exact=FALSE)
    d_ref_boot <- depth.(Z_ref_boot, Z_ref_boot, notion = method, exact=FALSE)

    ks_boots[b] <- get_ks_stat(d_samp_boot, d_ref_boot)
    cvm_boots[b] <- get_cvm_stat(d_samp_boot, d_ref_boot)
  }

  list(
    KS_p_value = mean(ks_boots >= ks_obs),
    CvM_p_value = mean(cvm_boots >= cvm_obs)
  )
}
```

```
)
}
```

Running the Tests on the dataset

We evaluate the p-values for both the KS and CvM tests. We reject the null hypothesis of multivariate normality if the p-value is less than our alpha level of 0.05.

```
res_mah <- depth_mvn_test(boston_clean, method = "Mahalanobis", B = 50)
print(res_mah)
```

```
## $KS_p_value
## [1] 0
##
## $CvM_p_value
## [1] 0
```

Final Conclusion

Both the Kolmogorov-Smirnov and Cramér-von Mises bootstrap tests yield p-values of 0 (or extremely close to zero, $p < 0.05$). Therefore, we heavily reject the null hypothesis H_0 . For complexity issues, we have used only 50 Bootstrap samples and for Mahalanobis only.

The combination of the severe curvature in all five DD-plots, the diverging Scale Curve, and the formal hypothesis test confirms with statistical significance that this subset of the Boston housing dataset does **not** follow a multivariate normal distribution. This is expected, as variables like crime rate (`crim`) are inherently highly right-skewed and do not behave symmetrically.

QUESTION 2

1. Introduction

The Depth-Based Decision Rule

Let the training data be denoted by $\chi = \{(X_1, y_1), (X_2, y_2), \dots, (X_n, y_n)\}$, where each $X_i \in \mathbb{R}^p$ is a p -dimensional feature vector and $y_i \in \{1, 2, \dots, k\}$ is the class label.

Given a new data point $X_{new} \in \mathbb{R}^p$, the depth-based classifier assigns it to the class y^* that maximizes the data depth relative to the distribution of that class:

$$y^* = \arg \max_{i \in \{1, \dots, k\}} \pi_i D(X_{new}, F_{y_i})$$

Where:

- $\chi_i = \{X_j : y_j = i\}$ is the set of training points belonging to class i .
- $D(X, F_{y_i})$ is the depth of point X with respect to distribution function F_{y_i} .
- y_i is repersent output of the i^{th} class.
- π_i is Prior Probability of class i

Probability of Misclassification (P_{err})

Based on the objective of minimizing the classification error, the theoretical probability of misclassification for a k -class problem is defined as:

$$P_{err} = \sum_{j=1}^k \pi_j P \left(\pi_j D(X, F_{y_j}) < \max_{i \neq j} \pi_i D(X, F_{y_i}) \mid X \in C_j \right)$$

For a two-class problem (C_1, C_2), as the sample sizes $n_1, n_2 \rightarrow \infty$, the classifier $g(n_1, n_2)$ converges to the theoretical misclassification probability g_0 :

$$g_0 = \pi_1 P[\pi_2 D(X, y_2) > \pi_1 D(X, y_1) \mid X \in C_1] + \pi_2 P[\pi_1 D(X, y_1) > \pi_2 D(X, y_2) \mid X \in C_2]$$

2. Mathematical Definitions

A. Mahalanobis Depth

Based on the squared Mahalanobis distance, it assumes an underlying elliptical distribution:

$$D_{MA}(x, \mu, \Sigma) = \frac{1}{1 + (x - \mu)^T \Sigma^{-1} (x - \mu)}$$

Mahalanobis depth is superior in the following specific cases:

1. Elliptically Symmetric Data
2. Small Sample Sizes in High Dimensions
3. Highly Correlated Features
4. Computational Efficiency:
5. Extrapolation and Out-of-Sample Prediction

B. Spatial Depth

A multivariate generalization of the median, calculating the average of unit vectors from x to all points z_i in the data cloud:

$$D_{SP}(x, P) = 1 - \left\| \frac{1}{n} \sum_{i=1}^n \frac{x - z_i}{\|x - z_i\|} \right\|$$

Spatial Depth is superior in the following specific cases:

1. Non-Elliptical Symmetry
2. Presence of Outliers (Robustness)
3. Heavy-Tailed Distributions
4. High-Dimensional Stability

C. Halfspace (Tukey) Depth

The minimum fraction of data points contained in any closed halfspace H that includes x :

$$D_{HS}(x, P) = \frac{1}{n} \inf_{\|u\|=1} \# \{i : u^T z_i \geq u^T x\}$$

Halfspace (Tukey) Depth is superior in the following specific cases:

1. Extreme Outliers (Maximum Robustness)
2. Non-Elliptical or Skewed Clusters
3. Affine Invariance
4. Large Sample Sizes ($N \gg p$)

D. Simplicial Depth

The probability that x is contained within a simplex formed by $d + 1$ random points from the dataset:

$$D_{SI}(x, P) = \frac{1}{\binom{n}{d+1}} \sum_{1 \leq i_1 < \dots < i_{d+1} \leq n} I(x \in S[z_{i_1}, \dots, z_{i_{d+1}}])$$

Halfspace (Tukey) Depth is superior in the following specific cases:

1. Small Dimensions ($p = 2$ or $p = 3$)
2. Non-Elliptical, Complex Geometries:
3. Robustness to Outliers

Data Loading

```
library(ddalpha)
data(iris) # Data Loading
```

Identify symmetry of the data

```
library(ggplot2)

elliptically_symmetric <- function(data_subset, species_name) {
  n <- nrow(data_subset)
  p <- ncol(data_subset)

  # Calculate depth to find the Multivariate Median
  depth_vals <- depth.spatial(as.matrix(data_subset), as.matrix(data_subset))
  # Per your notes: The observation with largest depth is the multivariate median
  center_point <- colMeans(data_subset)

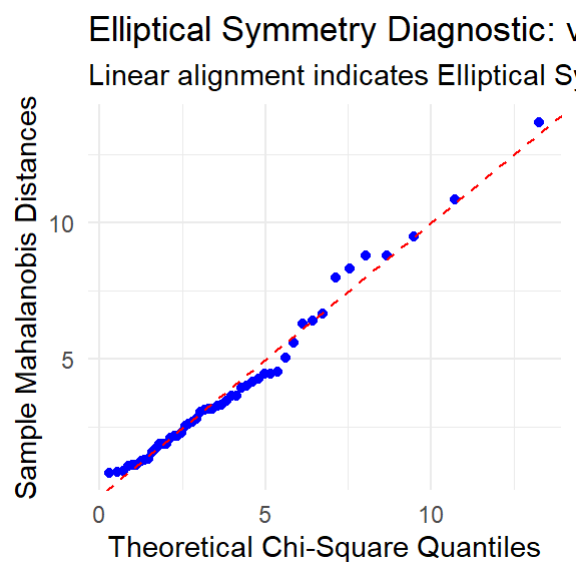
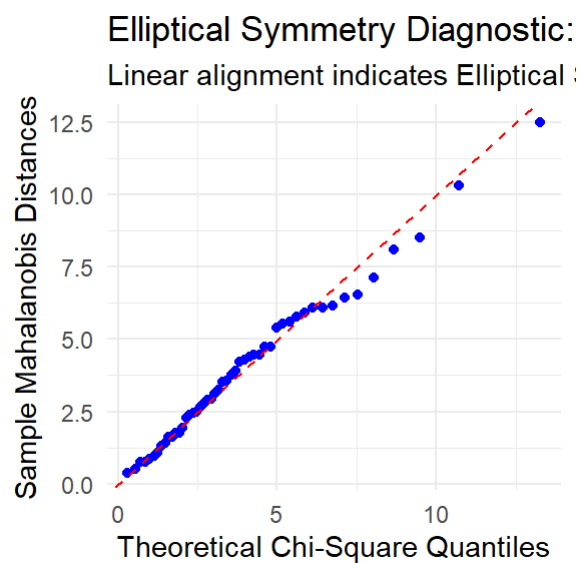
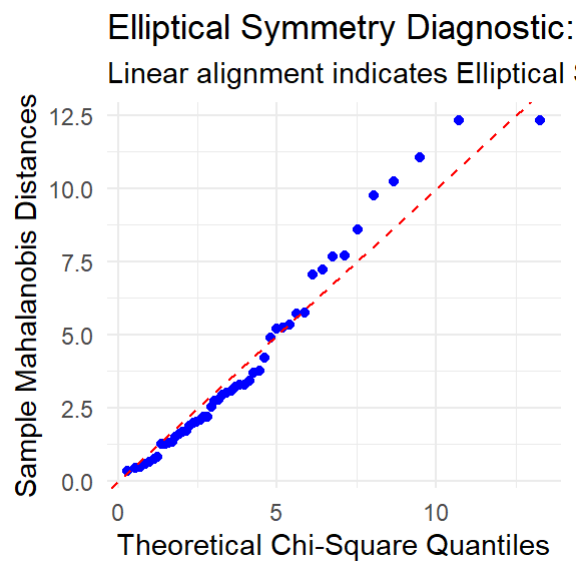
  # Calculation of Mahalanobis Distances ( $MD^2$ )
  S <- cov(data_subset)
  md2 <- mahalanobis(data_subset, center = center_point, cov = S)

  # Prepare Q-Q Plot Data
  expected_q <- qchisq(ppoints(n), df = p)
  observed_q <- sort(md2)
  df_qq <- data.frame(Theoretical = expected_q, Sample = observed_q)

  # Generate Plot
  ggplot(df_qq, aes(x = Theoretical, y = Sample)) +
    geom_point(color = "blue") +
    geom_abline(slope = 1, intercept = 0, color = "red", linetype = "dashed") +
    labs(title = paste("Elliptical Symmetry Diagnostic:", species_name),
         subtitle = "Linear alignment indicates Elliptical Symmetry",
         x = "Theoretical Chi-Square Quantiles",
         y = "Sample Mahalanobis Distances") +
    theme_minimal()
}

# Create a proper list of data frames
species_list <- list(
  setosa = iris[iris$Species == "setosa", 1:4],
  versicolor = iris[iris$Species == "versicolor", 1:4],
  virginica = iris[iris$Species == "virginica", 1:4]
)

# Loop and print plots
for(name in names(species_list)) {
  print(elliptically_symmetric(species_list[[name]], name))
}
```

Conclusion:- In all three plots, the sample Mahalanobis distances are not perfectly aligned with the theoretical chi-square quantiles, but some degree of alignment is visible. This suggests a certain level of elliptical symmetry in the data, which indicates that the data may approximately follow a multivariate normal distribution

Outlier Detection in Mulivariate setup

```
X <- iris[, 1:4] # Multivariate numeric features

mu <- colMeans(X)
S <- cov(X)

# Calculate Mahalanobis Distance ( $MD^2$ )
md_values <- mahalanobis(X, center = mu, cov = S)

# Threshold is 0.975
# In a  $p$ -dimensional normal distribution,  $MD^2$  follows a Chi-square
# distribution with  $p$  degrees of freedom.
p <- ncol(X)
cutoff <- qchisq(0.975, df = p)

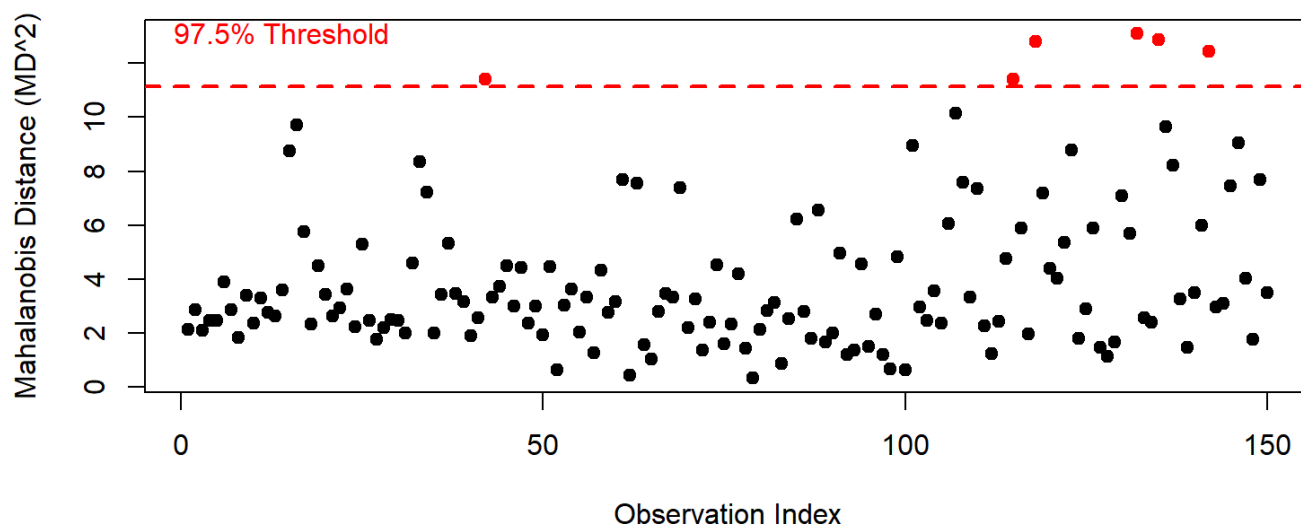
outlier_indices <- which(md_values > cutoff)
cat("Identified Outlier Indices:", outlier_indices, "\n")
```

```
## Identified Outlier Indices: 42 115 118 132 135 142
```

```
# Visualization: Distance Plot
plot(md_values,
     main = "Multivariate Outlier Detection (Mahalanobis)",
     xlab = "Observation Index",
     ylab = "Mahalanobis Distance ( $MD^2$ )",
     pch = 19, col = ifelse(md_values > cutoff, "red", "black"))

# Add the Chi-square threshold line
abline(h = cutoff, col = "red", lwd = 2, lty = 2)
text(x = 10, y = cutoff + 2, labels = "97.5% Threshold", col = "red")
```

Multivariate Outlier Detection (Mahalanobis)



Conclusion:- Only six observations (42, 115, 118, 132, 135, and 142) are identified as potential outliers, indicating that the dataset contains only a small proportion of extreme observations.

Data preparation for training and testing

```
X <- iris[,1:4] # input features
y <- iris$Species # Response variable

train_idx <- c(1:25, 51:75, 101:125)
test_idx <- c(26:50, 76:100, 126:150)

X_train <- as.matrix(iris[train_idx, 1:4])
y_train <- iris[train_idx, 5]

X_test <- as.matrix(iris[test_idx, 1:4])
y_test <- iris[test_idx, 5]

classes <- unique(y_train)
```

Training and test function

```
# =====
# 5. Depth-Based Classification (5 Methods)
# =====
methods <- c("halfspace", "Mahalanobis", "simplicial", "spatial", "simplicialVolume")
method_names <- c("Half-space (Tukey)", "Mahalanobis", "Simplicial", "Spatial (L1)", "Oja (Simplicial Volume)")

misclass_rates <- numeric(length(methods))
n_test <- nrow(X_test)
n_classes <- length(classes)

cat("\nCalculating depths and classifying... (Simplicial and Oja may take a few seconds)\n")
```

```
##
## Calculating depths and classifying... (Simplicial and Oja may take a few seconds)
```

```

for (m in seq_along(methods)) {
  method <- methods[m]
  depth_matrix <- matrix(0, nrow = n_test, ncol = n_classes)
  colnames(depth_matrix) <- as.character(classes)

  # Calculate depth of each test point w.r.t each training class
  for (i in 1:n_classes) {
    c <- classes[i]
    X_train_c <- X_train[y_train == c, ]

    # Unified wrapper for all depth functions
    depth_matrix[, i] <- depth.(X_test, X_train_c, notion = method)
  }

  # Assign to the class where the test point is "deepest"
  pred_indices <- max.col(depth_matrix, ties.method = "first")
  y_pred <- classes[pred_indices]

  # Record the misclassification rate
  misclass_rates[m] <- mean(y_pred != y_test)
}

# Print results table
results_df <- data.frame(
  Depth_Function = method_names,
  Misclassification_Rate = round(misclass_rates, 4),
  Accuracy = round(1 - misclass_rates, 4)
)

print("\n--- Classification Results ---")

```

```
## [1] "\n--- Classification Results ---"
```

```
print(results_df)
```

```
##           Depth_Function Misclassification_Rate Accuracy
## 1      Half-space (Tukey)           0.4400    0.5600
## 2           Mahalanobis           0.0267    0.9733
## 3           Simplicial           0.4667    0.5333
## 4           Spatial (L1)           0.0267    0.9733
## 5 Oja (Simplicial Volume)           0.0267    0.9733
```

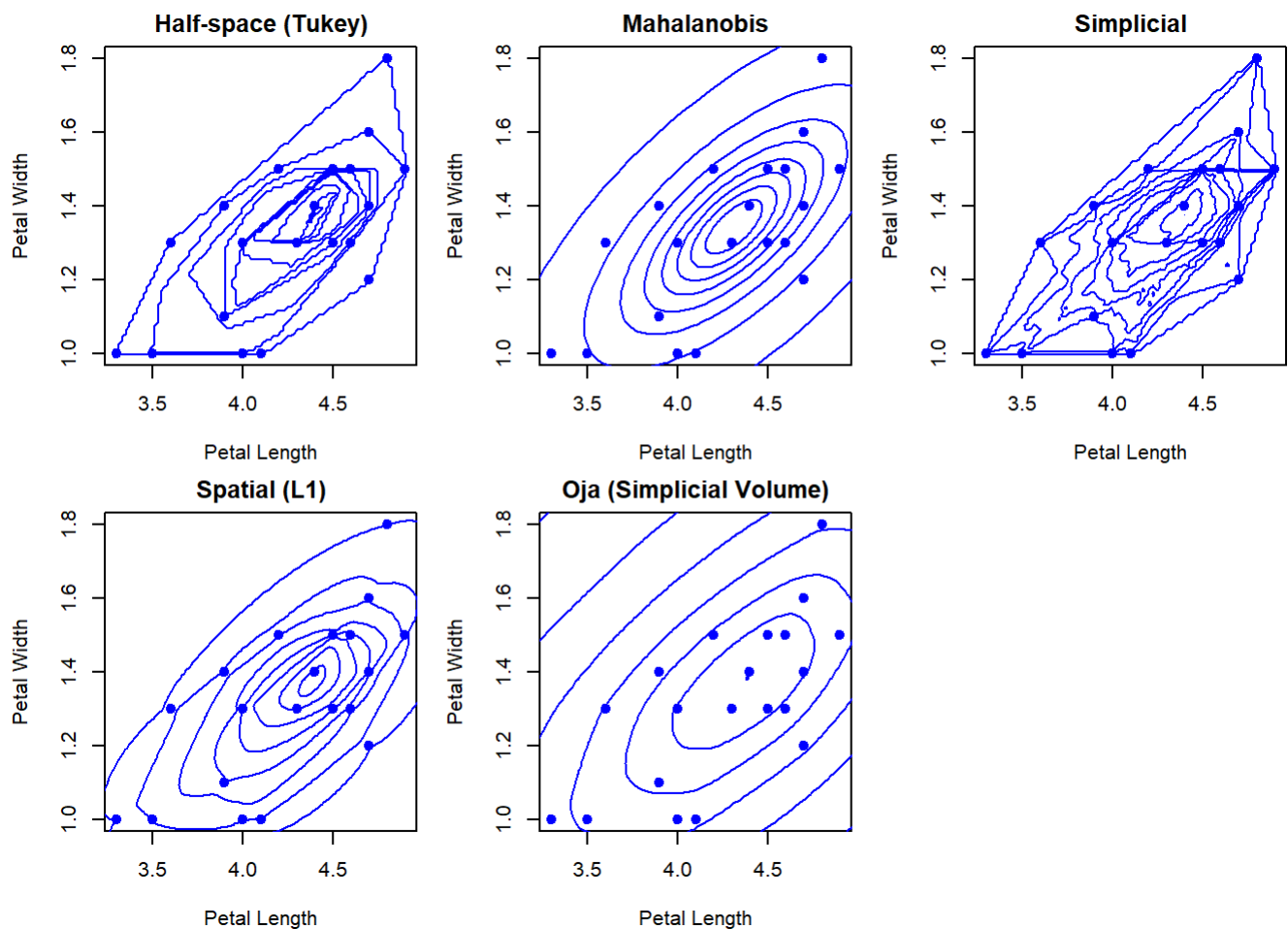
Visualizations

```
# Subset to Petal.Length and Petal.Width for 2D visual mapping
X_train_2D <- as.matrix(iris[train_idx, 3:4])
X_test_2D  <- as.matrix(iris[test_idx, 3:4])
X_train_vers_2D <- X_train_2D[y_train == "versicolor", ]
X_train_virg_2D <- X_train_2D[y_train == "virginica", ]

# --- Plot A: Compare Depth Contours ---
par(mfrow = c(2, 3), mar = c(4, 4, 2, 1))

for (m in seq_along(methods)) {
  depth.contours(X_train_vers_2D, depth = methods[m],
    main = method_names[m],
    xlab = "Petal Length", ylab = "Petal Width",
    col = "blue",
    exact = TRUE) # Prevents "static" noise on Oja/Simplicial
  points(X_train_vers_2D, pch = 16, col = "blue")
}

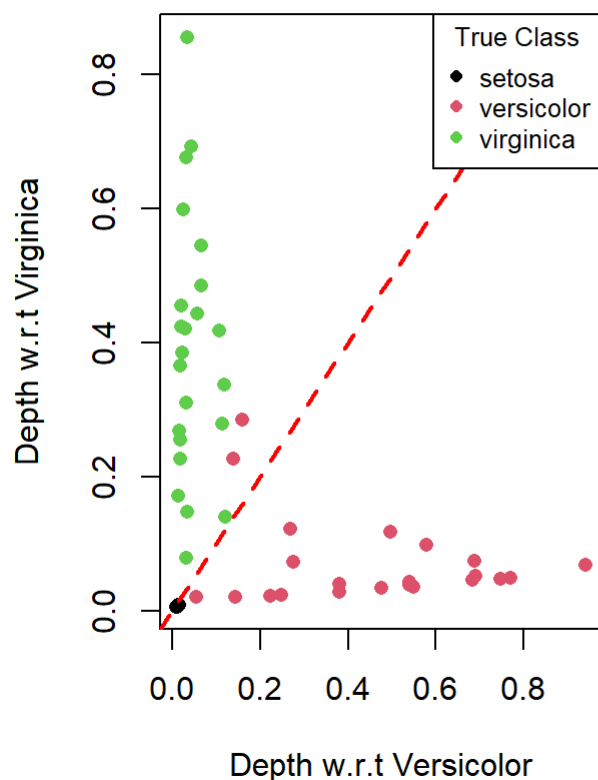
# --- Plot B: DD-Plot and Boxplots ---
par(mfrow = c(1, 2), mar = c(5, 4, 4, 2))
```



```
# 1. DD-Plot (Spatial Depth)
depth_wrt_vers <- depth.spatial(X_test_2D, X_train_vers_2D)
depth_wrt_virg <- depth.spatial(X_test_2D, X_train_virg_2D)

plot(depth_wrt_vers, depth_wrt_virg,
     col = as.numeric(as.factor(y_test)),
     pch = 16,
     xlab = "Depth w.r.t Versicolor",
     ylab = "Depth w.r.t Virginica",
     main = "DD-Plot (Spatial Depth)")
abline(a = 0, b = 1, lty = 2, col = "red", lwd = 2)
legend("topright", legend = levels(as.factor(y_test)),
      col = 1:3, pch = 16, title = "True Class", cex = 0.8)
```

DD-Plot (Spatial Depth)



Conclusion:-

1. The depth contour plots indicate that the observations follow an approximately elliptical pattern along the relationship between petal length and petal width. Methods such as Mahalanobis and Spatial depth clearly capture this elliptical structure, while the other depth measures show a similar central concentration of points. This suggests that the data exhibits an approximately elliptical distribution with most observations concentrated near the center and only a few potential outliers.
2. The DD-plot based on spatial depth shows that the observations from different classes exhibit some separation in the depth space, although there is partial overlap between versicolor and virginica. The setosa observations appear relatively more distinguishable compared to the other two classes. This indicates that spatial depth provides a reasonable but not perfect classification structure for the dataset.

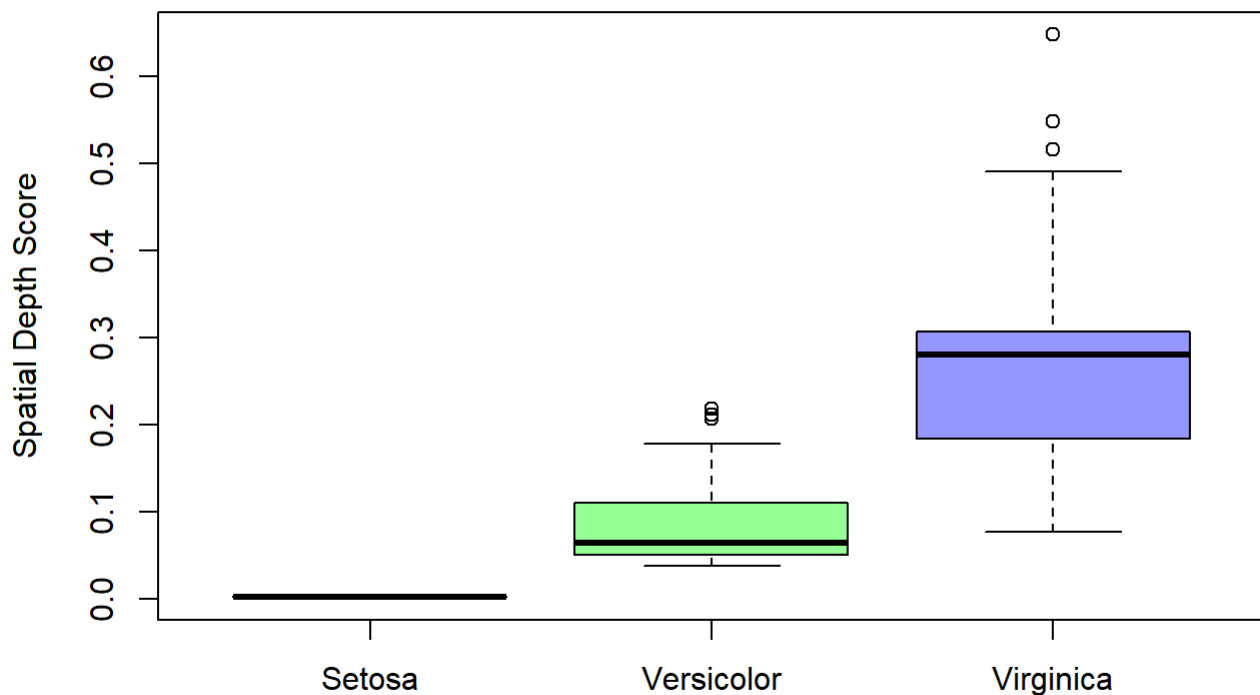
Depth Boxplots (Testing Spatial depth behavior for true Virginica points)

```
X_test_virg <- as.matrix(iris[126:150, 1:4])

depths_in_setosa <- depth.spatial(X_test_virg, as.matrix(iris[1:25, 1:4]))
depths_in_vers   <- depth.spatial(X_test_virg, as.matrix(iris[51:75, 1:4]))
depths_in_virg   <- depth.spatial(X_test_virg, as.matrix(iris[101:125, 1:4]))

boxplot(depths_in_setosa, depths_in_vers, depths_in_virg,
        names = c("Setosa", "Versicolor", "Virginica"),
        col = c("#FF9999", "#99FF99", "#9999FF"),
        main = "Depth of True Virginica Points",
        ylab = "Spatial Depth Score")
```

Depth of True Virginica Points



```
# Reset plotting layout to default
par(mfrow = c(1, 1))
```

Conclusion:- The boxplot shows that the spatial depth scores of true Virginica points are highest with respect to the Virginica class, indicating that these observations lie closer to the center of their own distribution. In contrast, the depth values with respect to Setosa and Versicolor are much lower. This suggests that spatial depth effectively captures class centrality and helps distinguish Virginica from the other classes.

Why Mahalanobis and Spatial performed better:

1. **Distribution Alignment:** The classes in the Iris dataset, particularly Setosa, are well separated and exhibit an approximately elliptical structure. Since Mahalanobis depth is designed to work effectively with data that follows an elliptical or multivariate normal distribution, it captures the underlying distribution of the data more accurately.
 2. **Effect of Outliers:** The dataset contains only a small number of outliers, so the overall covariance structure and central tendency of the data remain stable. As a result, methods such as Mahalanobis and Spatial depth, which rely on global distributional information, perform well and provide reliable classification results.
-